TIVIDY

Generic Case-Processing Workflow Management System

Final Report
COS 214: Software Modelling

University of Pretoria

01 November 2026

Group Members

Lindokuhle Skosana	**u24891968**
Joshua Peter	**u23626527**
Kristy Heesen	**u24702341**
LM Kabongo	**u25030338**
Musa Msibi	**u25098022**

Table of Contents

1. **Introduction**
2. **Workflow Management Research Findings**
3. **System Scope and Requirements**
4. **UML Activity Diagrams**
5. **Design Patterns**
6. **Class Diagram**
7. **Sequence and State Diagrams**
8. **Industry Demonstration A: Hospital Patient Admission**
9. **Industry Demonstration B: University Student Complaint**
10. **Verification Tests and Traceability**
11. **Build, Run and Test Instructions**
12. **GitHub, Workflow, and Team Contributions**
13. **Updates and Revisions**
14. **Conclusion**

References

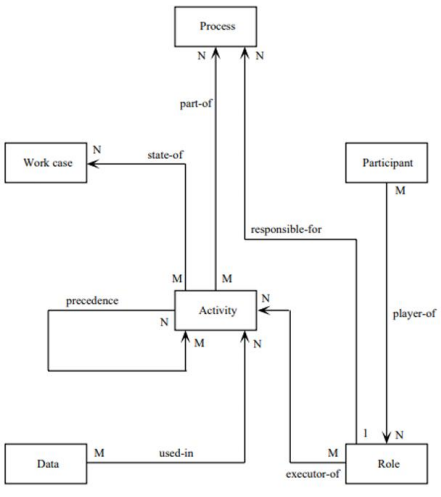


Figure 2.1 Workflow conceptual architecture

[2] Diagram of concept of a workflow architecture.

1. Introduction

TIVIDY is a generic case-processing workflow management system. Its purpose is to coordinate work across a case lifecycle without hard coding one organisation's business process into the engine. The same engine is demonstrated through two different scenarios: a hospital patient admission workflow and a university student complaint management workflow.

The implementation separates a reusable WorkflowDefinition from each WorkflowInstance created from it. The definition holds the configured workflow structure, assignment strategy and escalation handler. Each instance receives a cloned work tree and maintains its own state, task-progress records, assignments, participants and audit events. This separation is central to the project: one hospital admission or student complaint can progress without changing another case that uses the same definition.
The codebase contains twelve GoF pattern implementations across the three pattern categories: Builder and Prototype (creational); Adapter, Composite, Decorator and Facade (structural); and Chain of Responsibility, Command, Memento, Observer, State and Strategy (behavioural). The main TUI provides automatic and interactive versions of both industry demonstrations, as well as access to generated documentation. In interactive mode, users can select an instance, add or cancel work, alter a role or priority, reassign work, add a participant, start another instance, assign and complete tasks, and view that instance's audit history.

2. Workflow Management Research Findings

IBM defines a workflow as [1] a system for managing repetitive processes and tasks which occur in a particular order. They are the mechanism by which people and enterprises accomplish their work. Simply, a workflow is a repeatable sequence of steps that moves a process from start to finish. Workflow management systems are [2] systems designed to assist groups of people in carrying out work processes, and contain organizational knowledge of where work flows in the default case.

A workflow definition is [3] a template or blueprint for a process and a running workflow instance is [4] a single active execution of the template. Activities and work items are typically represented as [5] distinct nodes, data structures or objects that define and track how work moves in the system. Work items are generally assigned on a hierarchical basis where a "principal" (usually a customer or a boss) assigns a work item which are received can be passed down the hierarchy [6]. Approvals, rejections and escalations are usually based on human interaction, algorithms, timers or escalation events [7].

In a workflow management system participants, roles and responsibilities are typically represented by assigning people or systems to roles and then assigning those roles to workflow tasks [8]. Dependencies restrict activities until preceding work has been completed. Events act as triggers that mark when a workflow activity should begin or continue and failures are handled using defined methods such as error handling, retries or escalation. Workflows typically interact with external systems by exchanging data, triggering events and coordinating communication.

Workflow history and audit information is recorded through automatic logging that ensure that every step of a process is captured and traceable across a process [9]. Running workflows are monitored through real time data collection and tracking [10]. A workflow engine can be made to be reusable across different processes by being designed in such a way that the modelling and monitoring of tools, ways of interacting with resources and ways of different activities or work items interacting with each other all operate under the general case [11]. Simply put, it needs to separate the execution mechanics from specific business knowledge.

The workflow definition is static and therefore holds information about the structure of the workflow such as conditions, assigned roles and rules. A running instance holds information about one specific workflow such as its unique ID, current state and completed and pending tasks [3]. Real workflows model nested activities and sub-processes using hierarchies and rules where the parent's execution is paused until the child completes their task [6]. In a workflow system, business decisions are usually configurable (able to be changed to meet specific

requirements) such as routing decisions, approval rules, time limits, etc. Finally, changes can be made to workflow definitions while work is in progress by “workflow versioning” [12], this involves creating a branch from the public version, and publishing the new branch and storing the outdated branch in a version history.

The research indicates that TIVIDY should provide a generic mechanism for defining and executing workflows rather than implementing one fixed problem-solving process. It’s core concepts should include WorkItems, workflow definitions and instances, activities, participants roles, routing, states, events escalation, history and monitoring. These capabilities can then be used in different scenarios such as complaints, IT problems and work-task management, while the underlying system remains the same.

3. 1 System Scope and Requirements

TIVIDY is a generic case-processing workflow management system. It manages submitted cases through review, assignment, processing, decision-making, escalation and closure. The system provides reusable workflow features that allow it to be used by different organisations for a variety of problems.

The main responsibility of TIVIDY is to manage the movement of work. This is achieved through a structured process. It must be able to create and run workflow instances, manage work items, assign work to participants, track the state of work, apply routing and escalation rules, record workflow history, as well as determine when a workflow has been successfully completed.

TIVIDY is responsible for managing the workflow, not performing the real-world work itself.

This includes:

- Creating workflow definitions and instances
- Creating and managing work items
- Assigning work
- Tracking states
- Checking dependencies
- Routing work
- Escalating work
- Reacting to events
- Recording workflow history
- Determining when the workflow is complete

TIVIDY does not perform the actual work assigned to a participant. TIVIDY may also communicate with external systems. Different organisations may manage and operate their processes differently. Some parts of TIVIDY should, as a result, be able to change without requiring the workflow system to be redesigned.

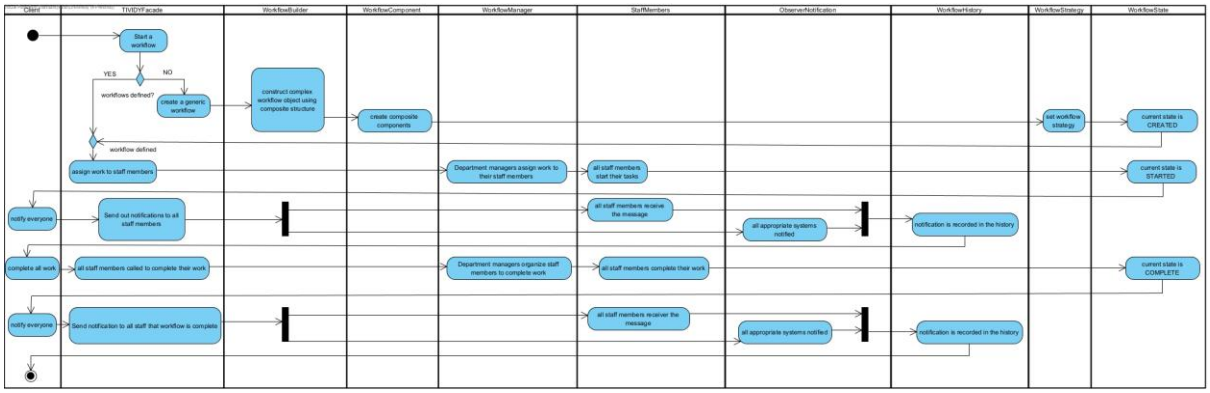
These may include:

- How work is assigned
- How escalation is handled
- How approvals are handled
- How the next work item is selected
- How workflow definitions are constructed
- How events are handled

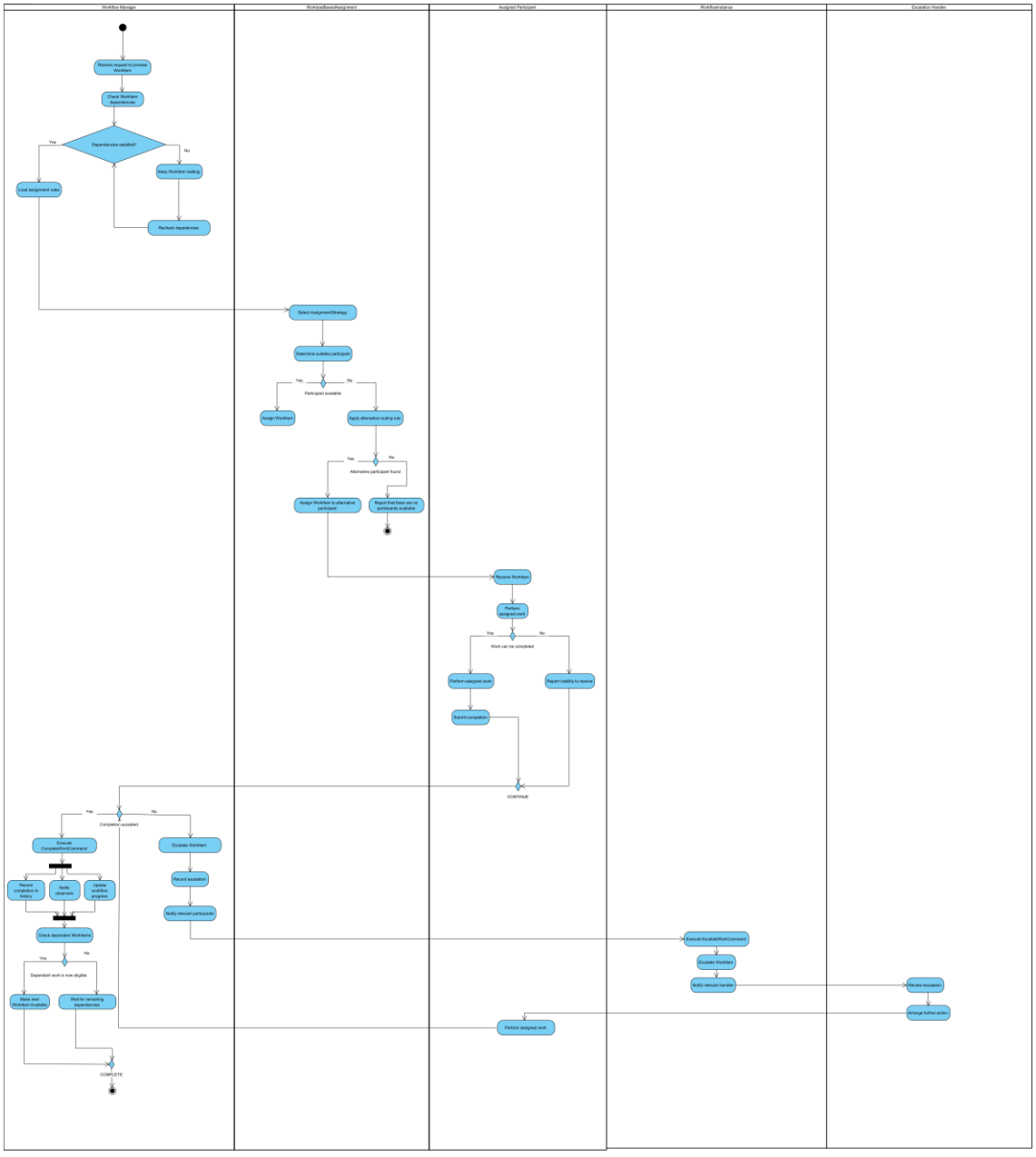
For example, one organisation may assign work according to a participant's role, while another organisation may assign work according to current workload.

4. UML Activity Diagrams

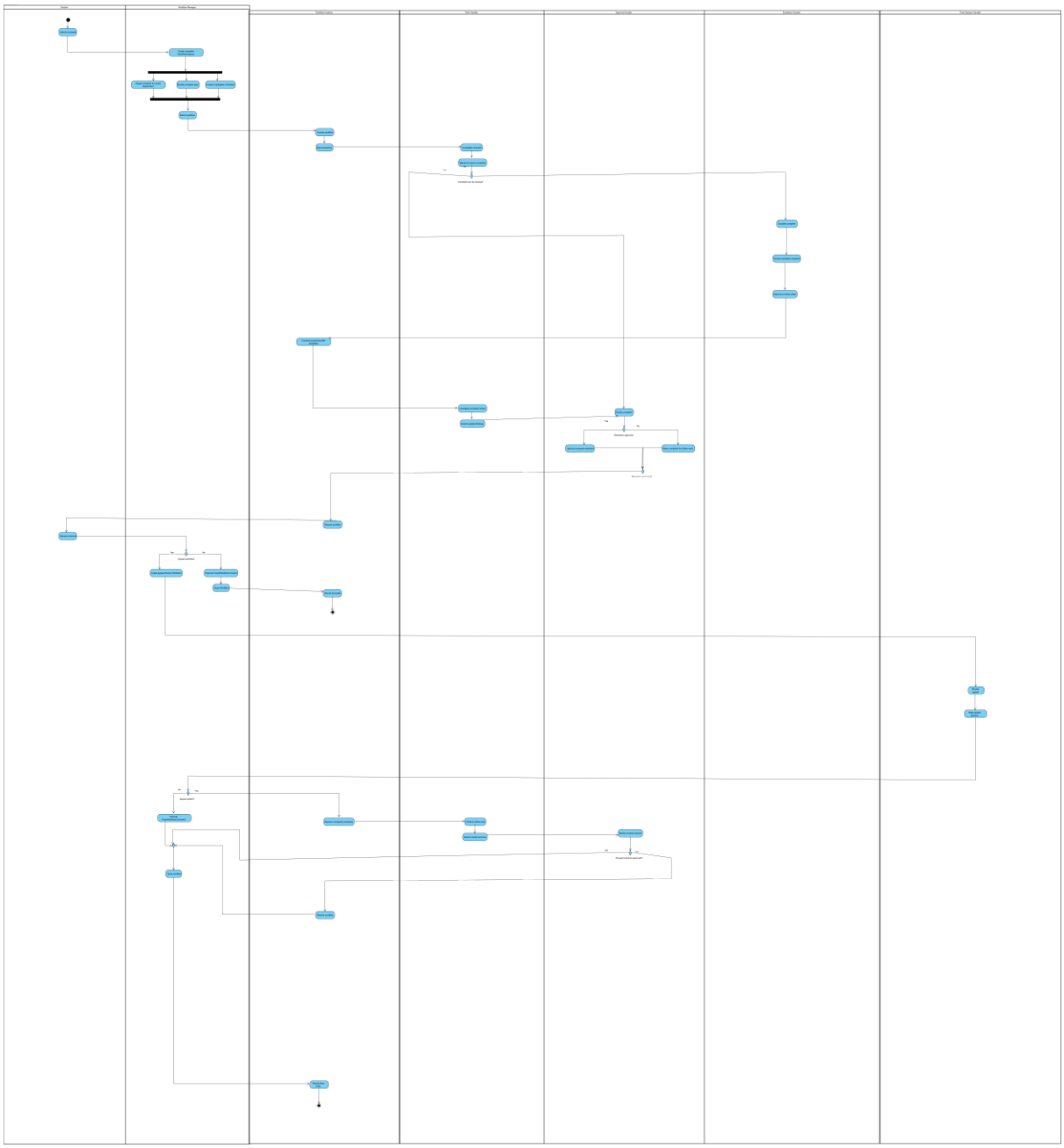
- Activity Diagram 1: Generic Workflow Execution



- Activity Diagram 2: Generic Work, Assignment, Completion and Escalation



- Activity Diagram 3: University Student Complaint Workflow



5. Design Patterns for TIVIDY

A. Creational Patterns

Builder:

Participants: WorkflowBuilder- (Builder), ConcreteWorkflowBuilder- (Concrete Builder), WorkflowDefinition- (product), WorkflowDirector is present for validation.

Problem solved: The two industry processes need definitions with different work items, dependencies, assignment strategies and escalation configuration. A builder provides incremental configuration without a large definition constructor.

Prototype:

Participants: WorkflowComponent- (Prototype), WorkItem, WorkflowStage, SubProcess and PriorityDecorator- (Concrete Prototypes), WorkflowDefinition supplies the root prototype, WorkflowInstance- (client/owner of the clone).

Problem solved: Many cases use the same definition but require independent completion flags and runtime work. Rebuilding every component for each case would duplicate configuration and risk inconsistent definitions.

B. Structural Patterns

Composite:

Participants: WorkflowComponent- (Component), WorkItem- (Leaf), WorkflowStage- and SubProcess- (Composites).

Problem solved: A workflow can contain individual work items as well as groups and nested subprocesses. Without a common interface, the engine would need separate logic for a single task and a group of tasks.

Decorator:

Participants: WorkflowComponent- (Component), WorkItemDecorator- (Decorator), PriorityDecorator- (Concrete Decorator).

Problem solved: A work component may need additional metadata or behaviour, such as priority, without creating a subclass for every possible combination of work item features.

Adapter:

Participants: NotificationService- (Target), EmailAdapter- (Adapter), ExternalEmailService- (Adaptee).

Problem solved: The engine should depend on a small notification interface rather than a specific external email API.

Façade:

Participants: TIVIDYFacade- (Facade), delegating to WorkflowDefinition, WorkflowInstance, WorkItem, Participant, assignment strategies and history.

Problem solved: The scenario code and terminal manager should not need to coordinate all the internal engine classes directly.

C. Behavioural Patterns

State:

Participants: WorkflowInstance- (Context), WorkflowState- (State), SubmittedState-, ActiveState-, InProgressState-, ResolvedState- and ClosedState- (Concrete States).

Problem solved: The operations allowed on a case depend on its lifecycle. Large conditional blocks in the Facade would mix lifecycle rules with work management.

Strategy:

Participants: AssignmentStrategy- (Strategy), RoleBasedAssignment- and WorkloadBasedAssignment- (Concrete Strategies), WorkflowDefinition-/Facade (clients).

Problem solved: Organisations can select suitable staff by role or by workload. The choice of selection algorithm should not be hard-coded into WorkItem.

Chain of Responsibility:

Participants: Handler- (Handler), ApprovalHandler-, EscalationHandler- and FinalDecisionHandler- (Concrete Handlers), WorkflowDefinition- configures the first handler.

Problem solved: Escalation requests may need to pass through several levels of review without the caller knowing which handler accepts the request.

Observer:

Participants: WorkflowInstance- (Subject), WorkflowObserver- (Observer), ProgressObserver-, HistoryObserver- and NotificationObserver- (Concrete Observers).

Problem solved: State transitions may have more than one interested component, such as progress display and audit recording. The instance should not directly call each observer's concrete class.

Command:

Participants: Command- (Command), AssignWorkCommand-, ApproveWorkCommand-, CompleteWorkCommand-, EscalateWorkCommand-, CancelWorkCommand- and CloseWorkflowCommand- (Concrete Commands), Invoker- (Invoker), WorkflowInstance- (Receiver).

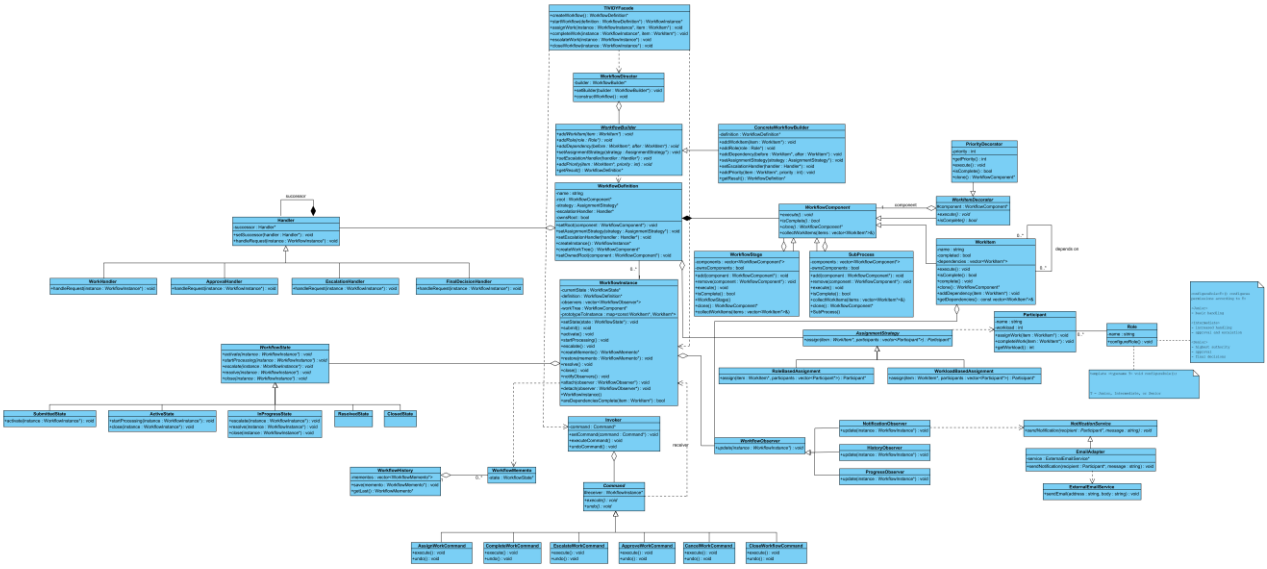
Problem solved: A workflow operation can be represented as an object so that the caller can execute or request undo without knowing the operation's implementation.

Memento:

Participants: WorkflowInstance- (Originator), WorkflowMemento- (Memento), WorkflowHistory- (Caretaker-like holder of memento pointers), Commands keep their own previous snapshot for undo.

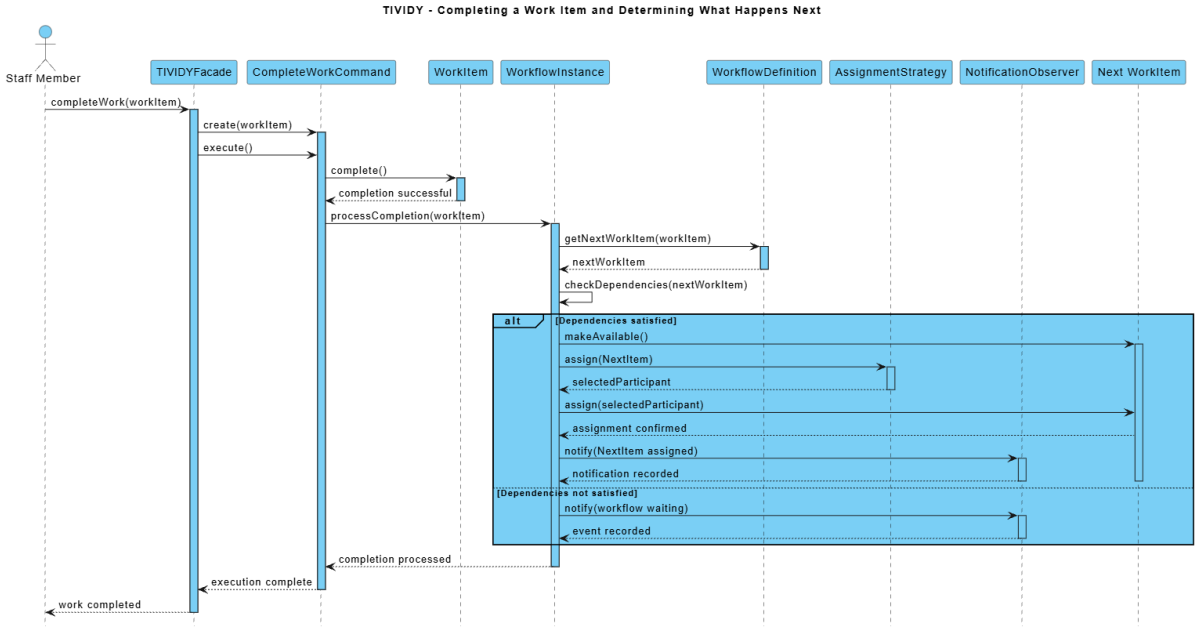
Problem solved: The command subsystem needs a way to retain a prior state without exposing the instance's internal state representation.

6. UML Class Diagram

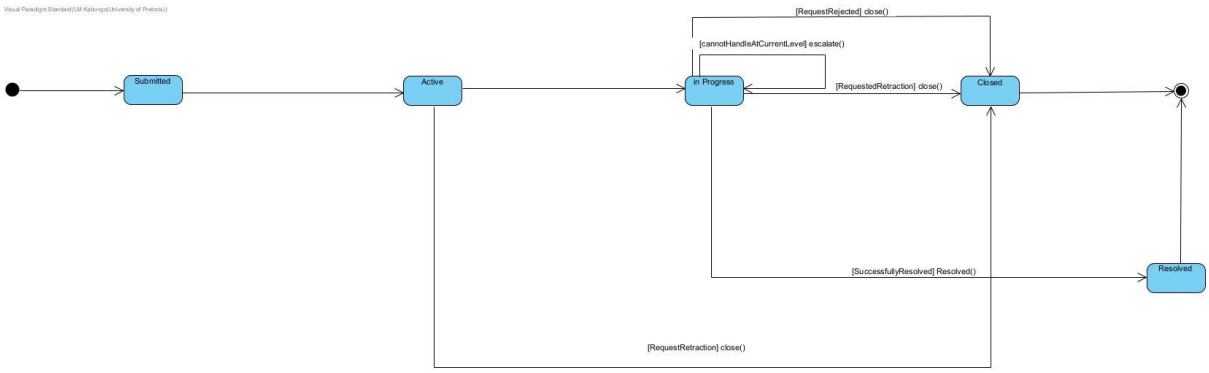


7. Sequence and State Diagrams

A. Sequence Diagram



B. State Diagram



8.Industry Demonstration A: Hospital Patient Admission

he hospital scenario coordinates the administrative admission of a demonstration patient. It is intended to illustrate a process with role-specific work, a conditional clinical-review step and a final admission/bed-allocation step. TIVIDY coordinates the sequence; hospital employees remain responsible for the actual verification, review, approval and allocation.

Work Items and Participants

Work Item	Required Role	Demonstration Participant
Register Patient	Receptionist	Nandi – Receptionist
Verify admission documents	Admissions Clerk	Peter – Admissions
Verify medical aid/payment	Building Officer	Ayesha – Billing
Emergency Clinical Review	Admissions Manager	Dr Mokoena – Admissions Manager
Approve Admission	Admissions Manager	Dr Mokoena – Admissions Manager
Complete Admission and allocate bed	Ward Coordinator	Thabo – Ward Coordinator

Process and routing

1. The user is asked whether the patient is an urgent admission.
2. Both routes register the patient, verify the admission documents and verify medical-aid/payment information in sequence.
3. For an urgent admission, the definition includes an Emergency clinical review dependency between payment verification and admission approval.
4. For a normal admission, the payment-verification task leads directly to admission approval; the emergency-review item is not inserted into the selected route.
5. Both routes then complete admission approval and bed allocation. When all active work items are complete, the Facade resolves the instance.

Failure paths and outcomes

If a task cannot be assigned because there is no suitable participant or a dependency has not been completed, the scenario treats the operation as a failure, cleans up the instance/definition and throws an explanatory runtime error. The Facade also rejects invalid assignments, null inputs and incomplete dependency paths. The demonstration does not simulate a real payment gateway failure, a missing bed, a clinical rejection, a retry timer or an external hospital-system outage; these are outside the current implementation. The expected demonstration outcome is a resolved admission after every required task is completed. The urgent route contains one additional review step. The case is not resolved by a medical decision computed by TIVIDY; it is resolved because the demonstration staff tasks are explicitly completed.

9. Industry Demonstration B: University Student Complaint

The university scenario coordinates a complaint by demonstration student , who disputes a recorded mark. The case is processed by faculty administration, a departmental representative, a faculty manager and student support. The goal is to show how TIVIDY can coordinate a complaint-like process that has sequential dependencies and an optional senior-review escalation

Work Items and Participants

Work Item	Required Role	Participants
Record student complaint	Faculty Administrator	Lerato – Faculty Office
Validate complaint and supporting evidence	Faculty Administrator	Lerator – Faculty Office
Investigate disputed mark	Department Representative	Mr Dlamini – Department
Review investigation outcome	Faculty Manager	Ms Naidoo – Faculty Manager
Notify student of outcome	Student Support Officer	Kabelo – Student Support
Close complaint case	Faculty Administrator	Lerato – Faculty Office

Process and escalation path

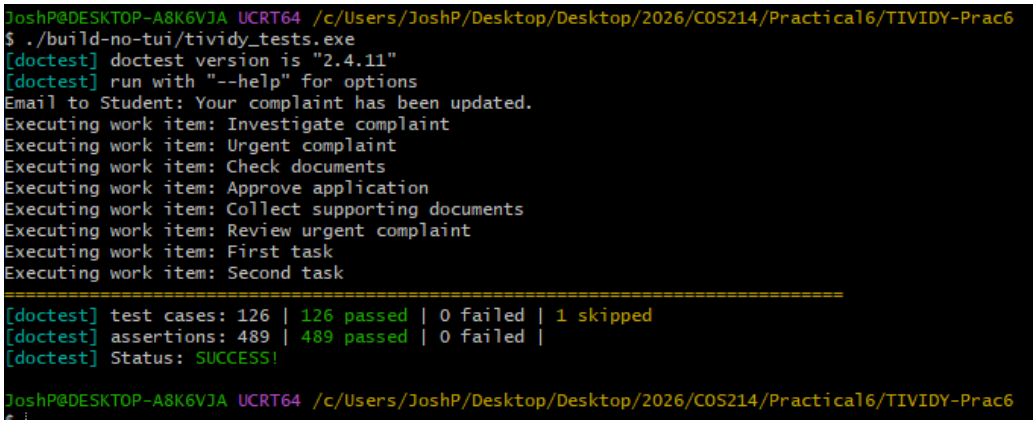
1. The faculty administrator records the complaint and validates the complaint and supporting evidence.
2. The departmental representative investigates the disputed mark. These three work items form a prerequisite chain.
3. The user is asked whether senior review is required.
4. If senior review is required, the Facade calls escalateWork(); the WorkflowInstance confirms that it is In Progress and sends the request through ApprovalHandler, EscalationHandler and FinalDecisionHandler.
5. The final handler accepts the request. The workflow remains In Progress: escalation does not introduce an EscalatedState or alter the work-item dependency sequence.
6. Whether or not escalation is requested, the workflow continues to review the outcome, notify the student and close the complaint case.
7. When all active work items are completed through the Facade, the instance transitions to Resolved.

Failure paths and outcomes

The scenario reports a failure if a required role has no suitable participant or a prerequisite is incomplete. Input failure while reading the senior-review decision also raises an exception. Escalation requires a configured handler; the engine throws if the definition has no handler. The normal and escalation routes both rejoin the same final review, notification and closure work sequence.

10. Verification, Tests and Traceability

Pattern-specific tests are present for Builder, Prototype, Composite, Decorator, Adapter, Facade, State, Strategy, Observer, Command, Memento and Chain of Responsibility. Additional integration tests exercise industry scenario modes, runtime workflow changes and a general smoke path. Specific requirements are intentionally tested demonstrating code to pattern to test traceability.



Requirement	Principal classes	Patterns	Test files
Definition/instance separation	WorkflowDefinition, WorkflowInstance	Builder, Prototype	prototype_test.cpp, facade_test.cpp
Reusable work hierarchy	WorkflowComponent; WorkItem; WorkflowStage; SubProcess	Composite	composite_test.cpp
Independent copies per case	clone(); createWorkTree(); prototypeToInstance map	Prototype; Composite	prototype_test.cpp; runtime_workflow_changes_test.cpp
Work assignment by configurable policy	AssignmentStrategy; RoleBasedAssignment; WorkloadBasedAssignment; TIVIDYFacade	Strategy; Facade	strategy_test.cpp; industry_scenario-modes-test.cpp
Dependency-aware progress	WorkItem::addDependency(); WorkflowInstance::areDependenciesComplete(); WorkItemProgress	Composite; Strategy	runtime_workflow_changes_test.cpp; smoke_test.cpp
State lifecycle and final states	WorkflowState; SubmittedState; ActiveState; InProgressState; ResolvedState; ClosedState	State; Observer	StateTests.cpp
Escalation routing	Handler; ApprovalHandler; EscalationHandler; FinalDecisionHandler	Chain of Responsibility	chain_test.cpp; industry_scenario-modes-test.cpp
State-change notifications	WorkflowInstance::attach(); detach(); notifyObservers(); ProgressObserver; HistoryObserver	Observer	observer_test.cpp; StateTests.cpp
Per-instance audit history	WorkflowHistory; AuditEvent; WorkflowInstance::getHistory(); TIVIDYFacade::getHistory()	Observer; Memento	runtime_workflow_changes_test.cpp; facade_test.cpp
Runtime modifications	WorkflowInstance::addWorkItem(); cancelWorkItem(); changeWorkItemRole(); changeWorkItemPriority(); assignParticipant(); TIVIDYFacade	Facade; Prototype	runtime_workflow_changes_test.cpp
External notification adaptation	NotificationService; EmailAdapter; ExternalEmailService	Adapter	adapter_test.cpp
Encapsulated operations and undo	Command; Invoker; concrete command classes; WorkflowMemento	Command; Memento	command_test.cpp; memento_test.cpp
Terminal scenario interaction	main.cpp; InteractiveWorkflowSession; TIVIDYFacade	Facade; Strategy; State	industry_scenario-modes-test.cpp; runtime_workflow_changes_test.cpp

11. Build, Run, Tests and Execution

On Ubuntu, install:

```
sudo apt-get update
sudo apt-get install -y build-essential cmake libncurses-dev git

git clone https://github.com/Kristy2005/TIVIDY-Prac6.git
cd TIVIDY-Prac6
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j2
./build/tividy
```

Engine and test build without Curses

On a machine where ncurses is not installed, configure the engine/tests-only build explicitly:

```
cmake -S . -B build-no-tui -DCMAKE_BUILD_TYPE=Debug -DTIVIDY_BUILD_TUI=OFF
cmake --build build-no-tui -j2
ctest --test-dir build-no-tui --output-on-failure
```

The matching Make targets are `make build-no-tui` and `make test-no-tui`. With the default option, CMake searches quietly for Curses; if it is unavailable, CMake skips the TUI executable and still builds the engine and tests. The CI workflow uses the default configure/build/test path and runs Valgrind; the supplied workflow file does not explicitly install libncurses-dev, so the TUI target is only built on runners where Curses is present.

Running tests, memory checks and documentation

make test

ctest --test-dir build --output-on-failure

make valgrind

make docs

`make docs` requires Doxygen and Graphviz. The CI configuration has a separate documentation job. The repository also provides a Dockerfile; from the repository root, build the image with `docker build -t tividy .`, launch the TUI with `docker run --rm -it tividy`, or run tests with `docker run --rm tividy ctest --test-dir build --output-on-failure`.

Running TIVIDY with Docker

TIVIDY can be built and executed inside a Docker container using Ubuntu 24.04 LTS.

Docker provides the required build tools and dependencies, including CMake, ncurses, Doxygen, Graphviz and Valgrind.

Prerequisites

- Docker Desktop installed and running
- Git installed to clone the repository

Build the Docker Image

From the root directory of the repository, run:

docker build -t tividy .

This builds the TIVIDY application and creates a Docker image named tividy.

Run TIVIDY

Run:

docker run --rm -it tividy

The -it flags enable interactive terminal input and output required by the terminal interface.

Run the Automated Tests

Run:

docker run --rm tividy ctest --test-dir build --output-on-failure

CTest executes the automated test suite and reports any failures.

Docker Configuration

The repository includes:

- Dockerfile - Defines the Ubuntu environment, installs dependencies and builds TIVIDY.
- .dockerignore - Excludes unnecessary files and generated build artefacts from the Docker context.

The application starts by default when the container is launched without another command.

12. GitHub Workflow and Team Contributions

Each member implemented their work on separate and dedicated branches. They all made sure to pull from main before any coding sessions and merged their changes into the main branch through pull requests that had to be verified by another member before being merged with main. No member ever coded directly on main, branches were named with clarity and understanding in mind and each member participated in the planning, implementation, documentation, testing and execution of the program.

Team Contributions

Task 1 - Research

Contributors: LM Kabongo, Joshua Peter

LM Kabongo and Joshua Peter worked on research into workflow-management systems and the concepts relevant to TIVIDY. This included workflow definitions and instances, activities, work items, assignment, routing, participants, dependencies, events, auditing, monitoring, reusability and workflow versioning.

Task 2 - Initial Scope and Scenario

Contributor: Kristy Heesen

Kristy Heesen developed the initial scope of TIVIDY and defined the University Student Complaint Management scenario. This included distinguishing workflow definitions from running workflow instances and establishing the responsibilities and boundaries of the generic workflow-management system.

Task 3 - UML Activity Diagrams

Contributors: Lindokuhle Skosana, Joshua Peter

Lindokuhle Skosana and Joshua Peter worked on UML Activity Diagrams for the generic TIVIDY workflows and the University Student Complaint Management scenario.

Task 4 - Design Patterns

Contributors: Kristy Heesen, Musa Msibi, Lindokuhle Skosana

Kristy Heesen, Musa Msibi and Lindokuhle Skosana worked on identifying and justifying the GoF design patterns used in TIVIDY. This included defining the design problems, pattern participants, collaborations, variation points and how the patterns work together within the workflow-management system.

Task 5 - UML Class Diagram

Contributor: Kristy Heesen

Kristy Heesen worked on developing the complete UML Class Diagram for TIVIDY. This included the main classes, relationships, multiplicities, attributes, operations, ownership relationships and collaborations between the different design patterns and subsystems.

Task 6 - Runtime Behaviour Diagrams

Contributors: LM Kabongo, Musa Msibi

LM Kabongo and Musa Msibi worked on the UML runtime-behaviour diagrams. LM Kabongo developed the sequence diagram showing completion of a WorkItem and determining what happens next, as well as the WorkflowInstance State Diagram. Musa Msibi developed the sequence diagram showing the escalation process.

Task 7 - Design Decisions and Revisions

Contributors: Entire Team

The team documented the major design decisions and revisions made throughout the practical, including changes to State, Chain of Responsibility, Command and other patterns as the design was refined.

Task 8 - GitHub, Testing and Documentation

Contributors: Entire Team

The team contributed to the GitHub repository, automated testing and project documentation, including research, UML diagrams, design decisions, the README and supporting project material.

13. Updates and Revision History

Throughout the design and implementation of TIVIDY, several parts of the original UML, architecture and pattern implementations were revised as the group gained a clearer understanding of the workflow system and received feedback. The main changes are documented below.

1. State Pattern Changed from WorkItem to WorkflowInstance

Problem / Original Design:

The original design applied the State pattern to individual WorkItem objects, with states such as Created, Available, Assigned, In Progress, Escalated, Resolved and Closed.

Change Made:

The State pattern was moved to WorkflowInstance, which now uses five concrete states:

- SubmittedState
- ActiveState
- InProgressState
- ResolvedState
- ClosedState

Reason for Change:

These states describe the lifecycle of an entire case rather than an individual task. Moving the pattern to WorkflowInstance provides a more accurate representation of the workflow lifecycle.

Diagram Updates:

The WorkItem–WorkItemState relationship was removed and replaced with the WorkflowInstance–WorkflowState relationship.

2. EscalatedState Removed

Problem / Original Design:

Escalation was originally represented as a separate state, requiring the workflow to move from InProgressState to EscalatedState and then back to InProgressState.

Change Made:

EscalatedState was removed. Escalation now occurs within InProgressState through the Chain of Responsibility, which locates a participant with sufficient authority.

Reason for Change:

Escalation changes who is responsible for handling a case, not necessarily the lifecycle stage of that case. Keeping escalation as an operation simplifies the state hierarchy.

Diagram Updates:

EscalatedState was removed from the State and Activity Diagrams. Escalation is represented as an action within the in-progress workflow.

3. Role and Participant Responsibilities Expanded

Problem / Original Design:

Role initially contained only a name, limiting its functionality and making the separation between Role and Participant less meaningful.

Change Made:

Role was expanded with a templated configureRole<T>() operation, with Junior, Intermediate and Senior configurations considered for different authority levels.

Participant was also expanded to support multiple roles, allowing one participant to perform different responsibilities.

Reason for Change:

Separating people from their roles makes the engine more reusable across different organisations. The same role can be shared by multiple participants, while a participant can hold several roles.

Diagram Updates:

The class diagram was updated to show Role separately from Participant and to represent their many-to-many relationship.

4. WorkflowDefinition and WorkflowInstance Separated

Problem / Original Design:

The original design did not sufficiently distinguish the reusable workflow structure from an individual execution of that workflow.

Change Made:

WorkflowDefinition represents the reusable blueprint, while WorkflowInstance represents one running case. Each instance maintains its own cloned work structure, progress, participants, assignments and history.

Reason for Change:

This allows multiple cases to use the same definition without their progress or modifications affecting one another. It is essential for a reusable workflow engine.

Diagram Updates:

The class diagram was revised to clarify the separate responsibilities and relationships of WorkflowDefinition and WorkflowInstance.

5. Creational Patterns Expanded: Builder and Prototype

Problem / Original Design:

The original design relied primarily on Builder and lacked a suitable mechanism for creating independent copies of workflow structures.

Change Made:

The Builder pattern was developed through WorkflowBuilder, ConcreteWorkflowBuilder and WorkflowDirector to construct workflow definitions.

The Prototype pattern was introduced through clone() operations on WorkflowComponent and its concrete implementations. WorkflowDefinition uses this functionality to create independent workflow structures for new instances.

Reason for Change:

Builder separates workflow construction from execution, while Prototype allows existing workflow structures to be cloned instead of rebuilt manually. Together, they support flexible construction and independent workflow instances.

Diagram Updates:

The relationships between the builders, WorkflowDefinition and the cloneable WorkflowComponent hierarchy were added or clarified.

6. Composite Pattern and Work Dependencies Improved

Problem / Original Design:

The original workflow structure was relatively simple and did not sufficiently represent nested processes or dependencies between tasks.

Change Made:

The Composite pattern was implemented using WorkflowComponent, WorkItem, WorkflowStage and SubProcess. Work items can also have dependencies that must be completed before they become available.

Reason for Change:

This allows workflows to contain individual tasks, groups of tasks and nested processes through a common interface. Dependencies ensure that work is completed in the correct order.

Diagram Updates:

The class diagram was updated to show the Composite hierarchy and its relationships.

7. Assignment Strategy and Participant Workloads Improved

Problem / Original Design:

Assignment behaviour was limited and risked becoming tightly coupled to the workflow engine.

Change Made:

The Strategy pattern was introduced through AssignmentStrategy, RoleBasedAssignment and WorkloadBasedAssignment. Participants also maintain information about their assigned work and workloads.

Reason for Change:

Different organisations may require different assignment methods. Strategy allows assignment behaviour to change without modifying the core workflow engine, while workload tracking supports more balanced task distribution.

Diagram Updates:

The class diagram was updated to show the assignment strategy interface, its implementations and their relationship with WorkflowInstance and Participant.

8. History and Observer Patterns Expanded

Problem / Original Design:

The original history implementation mainly stored workflow state Mementos and did not provide a sufficiently descriptive record of actions performed during a case.

Change Made:

WorkflowHistory was expanded to store timestamped AuditEvents alongside WorkflowMementos. Each WorkflowInstance maintains its own history.

The Observer pattern was also clarified, with ProgressObserver, HistoryObserver and NotificationObserver having separate responsibilities. State-change notifications were centralised in WorkflowInstance.

Reason for Change:

This provides a clearer record of workflow progress and actions while separating notification and monitoring responsibilities from the main engine.

Diagram Updates:

The class diagram was updated to show the history, audit-event and observer relationships.

9. Command, Memento, Decorator and Adapter Patterns Refined

Problem / Original Design:

The engine needed a consistent way to execute and undo workflow operations, extend work items with additional information and communicate with external services.

Change Made:

The Command pattern encapsulates operations such as assigning, completing, cancelling and escalating work. The Memento pattern allows previous workflow lifecycle states to be restored.

The Decorator pattern allows additional properties, such as priority, to be associated with workflow components. The Adapter pattern connects TIVIDY's NotificationService interface to an external email service through EmailAdapter.

Reason for Change:

These patterns reduce coupling and allow functionality to be extended without unnecessarily modifying the core workflow classes. They also make workflow operations easier to manage and test.

Diagram Updates:

The class relationships between commands, Mementos, decorators and notification adapters were clarified.

10. Facade and TUI Expanded to Support Runtime Management

Problem / Original Design:

The original TUI mainly executed predefined workflow scenarios and did not provide sufficient control over running instances.

Change Made:

TIVIDYFacade was expanded to provide a central interface for managing definitions, instances, work items, participants, assignments and history.

A shared live-management menu was also introduced for both industries. It allows users to add, cancel and alter work, add participants, start additional instances and inspect audit history.

Changes apply to the selected instance only, leaving the original definition and other instances unchanged.

Reason for Change:

The Facade simplifies interaction with the engine, while runtime management demonstrates that TIVIDY can handle changing requirements during workflow execution rather than only following predefined scenarios.

Diagram Updates:

A runtime workflow-management sequence diagram was added to show the interaction between the user interface, Facade and WorkflowInstance.

11. Build Configuration and Documentation Improved

Problem / Original Design:

The original build configuration required Curses, making it difficult to compile the engine and tests on systems without the required terminal libraries. The README also needed more complete setup and usage instructions.

Change Made:

CMake was updated to make the TUI optional, allowing the engine and tests to build independently of Curses.

The Makefile and README were also expanded to provide build, run and test commands, dependency information, TUI instructions, design-pattern explanations, repository structure, Docker instructions and troubleshooting information.

Reason for Change:

Separating the engine from the TUI improves portability and allows the core functionality to be tested independently. The updated documentation also makes the project easier to build, understand and demonstrate from a fresh repository clone.

Diagram Updates:

A runtime-management sequence diagram was added, and the existing diagrams were intended to be aligned with the final implementation.

Conclusion:

TIVIDY demonstrates how a generic workflow engine can coordinate materially different case processes without encoding either industry into the engine itself. Its two scenarios share the same definition/instance model, work-component hierarchy, assignment interface, lifecycle states, escalation extension point, audit-history structure and Facade. The interactive manager makes the distinction between a reusable definition and an individual case visible by allowing a user to modify one instance and start another from the unchanged definition. The design uses twelve GoF pattern implementations to separate construction, cloning, work structure, optional features, assignment, lifecycle, event reaction, escalation, notification adaptation, command execution and access to the subsystem.

sReferences:

1. IBM (2026a) What is a workflow? IBM Think. Available at: https://www.ibm.com/think/topics/workflow (Accessed: 2 October 2026).
2. Ellis, C.A. (1999) 'Workflow Technology', in Beaudouin-Lafon, M. (ed.) Computer Supported Cooperative Work. Chichester: John Wiley & Sons. Available at: https://www.lri.fr/~mbl/cgi-bin/getpdf.cgi?Trends-CSCW/chap2.pdf (Accessed: 2 October 2026).
3. IBM (2026b) Workflow definitions and workflow instances. IBM OpenPages Documentation. Available at: https://www.ibm.com/docs/en/openpages/9.2.x?topic=fundamentals-workflow definitions-workflow-instances (Accessed: 2 October 2026).
4. TMS Software (2025) Basic concepts. TMS Workflow Studio Documentation. Available at: https://doc.tmssoftware.com/biz/workflow/guide/basic.html#workflow-instance (Accessed: 2 October 2026).
5. IBM (2026c) Workflow elements. IBM Documentation. Available at: https://www.ibm.com/docs/en/sig-and-i/11.0.2?topic=planning-workflow-elements (Accessed: 2 October 2026).
6. Aalst, W.M.P. van der and Hee, K.M. van (2001) Workflow Management: Models, Methods, and Systems. Eindhoven: Eindhoven University of Technology. Available at: https://pure.tue.nl/ws/files/2456322/543561.pdf (Accessed: 2 October 2026) [page 21 – "Allocating and accepting work"].
7. Camunda (2026) User tasks. Camunda 8 Documentation. Available at: https://docs.camunda.io/docs/components/modeler/bpmn/user-tasks/ (Accessed: 2 October 2026).
8. IBM (2026d) Workflow processes and user responsibilities. IBM Documentation. Available at: https://www.ibm.com/docs/en/max-it/cd.0.0_cd?topic=components-workflow-processes user-responsibilities (Accessed: 2 October 2026).
9. Hyperbots (2026) Workflow history. Available at: https://www.hyperbots.com/glossary/workflow-history (Accessed: 2 October 2026).
10. SearchInform (2026) Workflow management: Monitoring. Available at: https://searchinform.com/articles/cybersecurity/measures/security-monitoring/workflow management/monitoring/ (Accessed: 2 October 2026).
11. Ferreira, D. (2004) Developing Workflow Management Systems. Available at: https://web.ist.utl.pt/diogo.ferreira/papers/ferreira04developing.pdf (Accessed: 2 October 2026).
12. JetAdmin (2026) Workflow versioning. Available at: https://www.jetadmin.io/ (IBM, 2026)blog/workflow-versioning/amp/ (Accessed: 2 October 2026).